Vulnerability Report — SQL Injection (Authentication Bypass → Unauthorized Admin Action)

Status	Done
Category	Challenge
Keywords	Authentication Bypass SQL Injection
Date	@17/10/2025
Target	`POST /login` (authentication endpoint) → Admin loan management UI
Severity	Critical

Summary

I submitted a loan as a regular user. Later I bypassed authentication via a SQL injection in the login flow, gained admin access, approved the loan, and confirmed the approval in the user view. This report covers what happened, the impact, root cause, and evidence. Reproduction details and payloads are omitted and restricted to authorized testing environments.

Key timelines

- Logged in as a regular user and submitted a loan request.
- Attempted to log in again using input that triggered an SQL injection at `POST /login`.
- Bypassed authentication and gained an administrative session.

- Approved the pending loan in the admin UI.
- Returned to the original user account and verified the loan was **approved**.

Impact

- Complete account takeover, including admin privileges.
- Unauthorized financial action: a loan was approved without proper permission.
- Data integrity risk: important business records were changed by someone who shouldn't have had access.
- Confidentiality and compliance risk: sensitive customer financial data may have been exposed or altered.
- Business impact: risk of fraudulent approvals, financial loss, and harm to the company's reputation.

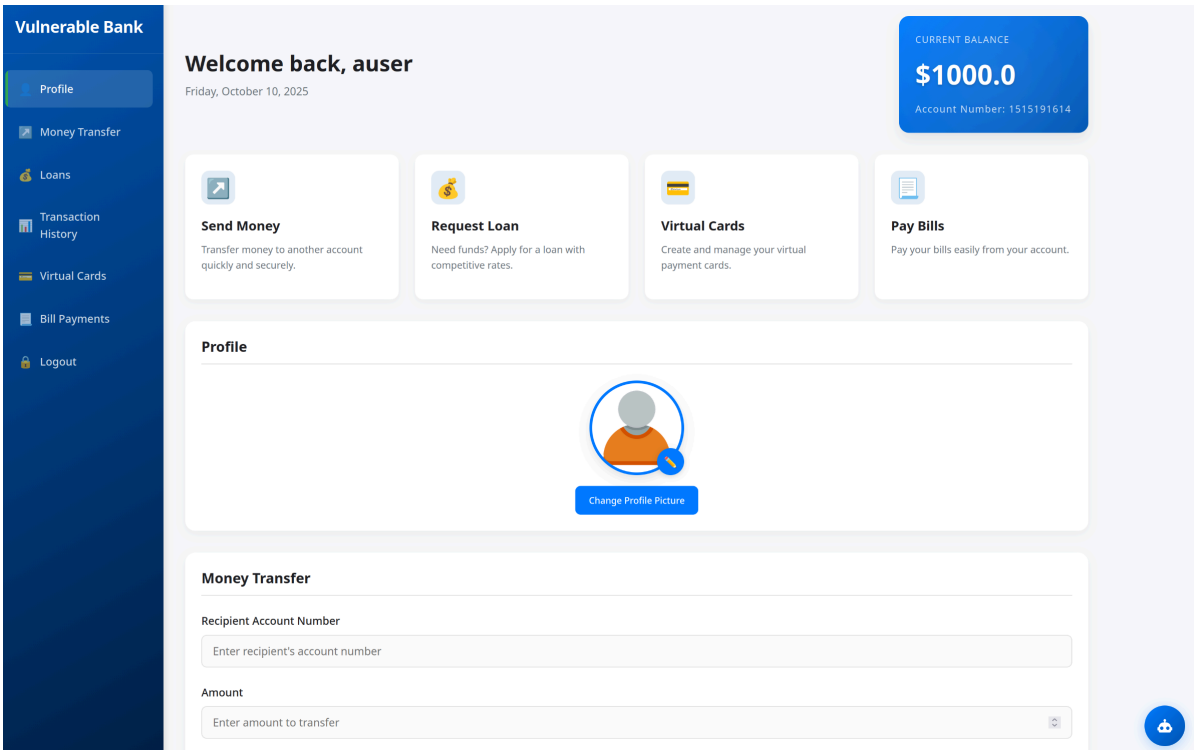
Root cause

The login code builds SQL statements by sticking user input directly into the query, which makes it possible for an attacker to change the query's meaning and log in without a valid password. Password checks can be manipulated because the attacker can change the `WHERE` clause. The app also records the raw input in server logs (including the attack data), which makes it harder to investigate safely.

Evidence

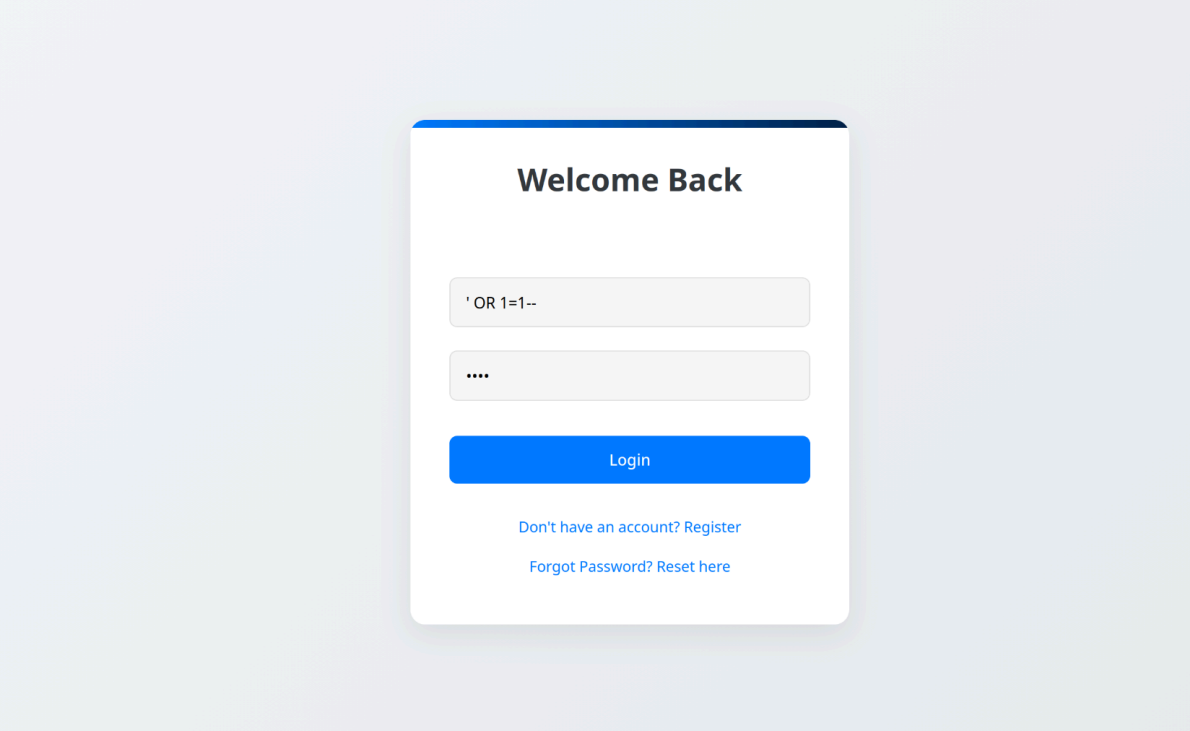
1. **Login attempt & server log**
   Shows server constructing and running a concatenated SQL query for authentication (raw user input visible in the log).


Vulnerable Bank
Guide
API Docs
Banking Made Simple & InSecure
This is an Intentionally vulnerable application, designed for everyone to practice application security.
Login
Register
VULN-BANK
Banking in Your Pocket
Experience the future of mobile banking with our cutting-edge vulnerable android app
Access all your banking needs on-the-go. Transfer money, pay bills, and manage your finances with just a few taps. Security and convenience, redefined.
Download
Vulnerable Bank
Total Balance
$12,450
Send
Pay
History


Vulnerable Bank
Profile
Money Transfer
Loans
Transaction History
Virtual Cards
Bill Payments
Logout
Welcome back, auser
Friday, October 10, 2025
CURRENT BALANCE
$1000.0
Account Number: 1515191614
Send Money
Transfer money to another account quickly and securely.
Request Loan
Need funds? Apply for a loan with competitive rates.
Virtual Cards
Create and manage your virtual payment cards.
Pay Bills
Pay your bills easily from your account.
Profile
Change Profile Picture
Money Transfer
Recipient Account Number
Enter recipient's account number
Amount
Enter amount to transfer

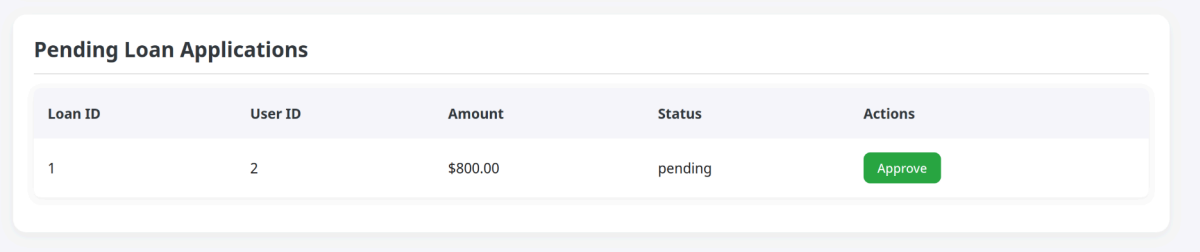
Server log showing the `SELECT * FROM users WHERE username='...' AND password='...'` pattern and the login attempt.

2. **Login form (injection input visible)**

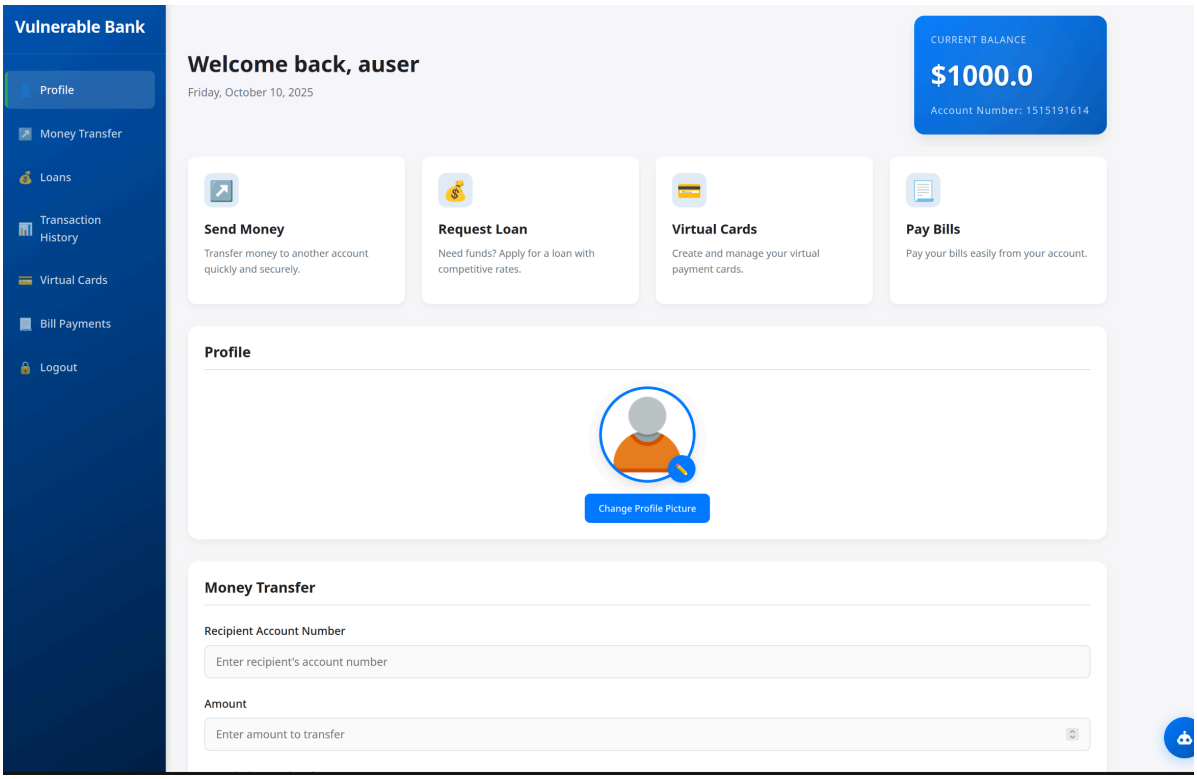


3. **Admin UI — Pending Loan Applications (post-approval)**

Shows the pending loan list with the loan shown as approved after the admin action.



4. **Result**



Immediate Actions

- **Log out all admin users** and block any suspicious sessions.
- **Save logs and system state** for investigation — don't restart or delete anything until backed up.
- **Change database passwords** and give DB accounts only the access they need.
- **Temporarily disable the login page**, using a firewall rule or maintenance mode.
- **Undo or review the loan approval** and any other admin actions during the breach.
- **Alert your internal teams** — security, legal, compliance, and incident response.

Short-Term Fixes

- **Use parameterized queries** — don’t build SQL with user input directly.
- **Hash all passwords** using a secure algorithm (e.g., bcrypt or Argon2).
- **Check passwords in code**, not inside SQL queries.
- **Sanitize and validate all user input** on the server.
- **Stop logging sensitive input**, especially raw passwords or attack data.

Long-Term Improvements

- **Use least-privilege access** for all database and app accounts.
- **Enable MFA** for admins and sensitive actions.
- **Add alerts** for unusual admin activity.
- **Require two-person approval** for high-risk actions like loan approvals.
- **Improve logs**: keep what’s needed for forensics, but don’t store sensitive inputs.

Verification Plan

1. Update the code to use safe SQL and secure password handling.
2. Test in staging with both automated and manual security checks.
3. Have security leads approve and perform a retest.
4. Monitor live systems for unusual logins or admin actions.
5. Run a post-incident review and update policies and coding standards.